

Computer Vision and Natural Language Processing Project

CSE555 Final Project — Group 5: Saksham Lakhera & Ahmed Zaher

Image Classification CV and Fine-Tuned NLP Recipe Recommendation

Authors: Saksham Lakhera and Ahmed Zaher

Date: July 2025



Download Project PDF

1. Abstract

The project is a **recipe recommendation system** that allows users to either **type a textual query** or **upload images of food items**. Based on the inputs, including user-provided tags and detected ingredients, the application returns the most relevant recipes using semantic search and image classification.

1.1 NLP Task:

This project addresses the challenge of improving recipe recommendation systems through advanced semantic search capabilities powered by transformer-based language models.

We fine-tune BERT (Bidirectional Encoder Representations from Transformers) to capture domain-specific context and understand nuanced relationships between ingredients and cooking techniques.

A subset of 15,000 recipes was preprocessed and structured into sequences categorized by food components (proteins, vegetables, grains, etc.) to optimize BERT input.

The model learns contextual embeddings that capture semantic meaning between ingredients and tags. Once trained, we generate embeddings for all recipes and use **cosine similarity** to retrieve the top-K relevant recipes for a user query.

1.2 CV Task:

In parallel, the computer vision component focuses on recognizing food items from images using deep learning.

We implemented an image classification pipeline based on **EfficientNet-B0**, trained to classify four distinct food categories: **Onion, Strawberry, Pear, and Tomato**.

In addition to identifying the type of produce, the model also detects **intra-class variations**, such as whether the item is **whole, halved/hulled, or sliced/cored**.

EfficientNet-B0 was chosen for its small size, pretraining on ImageNet (which includes visually similar classes), and ease of deployment. With minimal fine-tuning, it delivered high accuracy in both produce and variation classification tasks.

The goal was to evaluate both **inter-class** and **intra-class** visual consistency using statistical analysis and CNN-based classification.

Since the dataset was **manually created**, image analysis helped us understand variation across samples, identify noise, and decide on preprocessing techniques and model input parameters.

Together, both the NLP and CV pipelines form a **multimodal system** that enables recipe recommendations from either **text queries** or **food images**, offering a seamless and intelligent user experience.

2. Introduction

In an increasingly digital culinary world, users often look for personalized recipe recommendations based on either what they have at hand or what they crave. While traditional recipe search engines rely heavily on keyword matching, they fail to understand the deeper semantic context of ingredients, cooking methods, and dietary preferences. Similarly, visual recognition of food items can play a key role in enabling intuitive, image-based search experiences, especially for users unsure of ingredient names or spelling.

This project aims to build an **end-to-end multimodal recipe recommendation system** that supports both **natural language queries** and **image-based inputs**. Users can either type a textual query such as “healthy vegetarian salad” or upload an image of a food item (e.g., pear, onion), and the system will return the most relevant recipes. This is achieved by integrating two advanced deep learning pipelines:

- An **NLP pipeline** that fine-tunes a BERT model to capture culinary semantics and perform semantic recipe retrieval.
- A **CV pipeline** that classifies food items and their variations (e.g., whole, sliced) using EfficientNet-B0.

The project serves not only as a technical showcase of how language and vision models can be combined for real-world tasks, but also as an **educational exercise** that provided the team with hands-on experience in data preprocessing, model training, evaluation, deployment, and user interface design.

Ultimately, the system demonstrates how domain-specific adaptation of existing state-of-the-art models can lead to an intelligent and user-friendly solution for everyday tasks like recipe discovery.

3. CV: Produce Classification Task

For our Produce Classification task, we manually captured images of **tomato, onion, pear, and strawberry**, collecting a total of **12,000 images**, approximately **3,000 per category**.

Within each category, we introduced **3 intra-class variations**, with around **1,000 samples per variation**, by photographing the produce in different physical states:

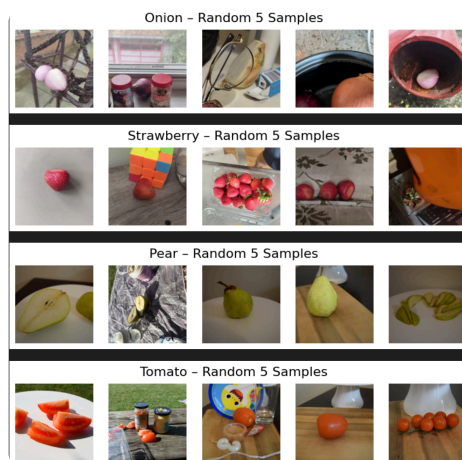
- **Whole:** The item is uncut and intact (e.g., an entire pear or onion).
- **Halved/Hulled:** The item is partially cut, for example, a strawberry with the hull removed or a fruit sliced in half.
- **Sliced:** The item is cut into smaller segments or slices, such as onion rings or tomato wedges.

These variations allow the model to generalize better by learning visual features across different presentations, shapes, and cross-sections of each produce type.

3.1 Image Preprocessing and Samples

As we are using the EfficientNet-B0 model, all images in our dataset are resized to **224×224** pixels. This is the standard input size for EfficientNet and ensures compatibility with pre-trained weights as well as efficient GPU usage during training.

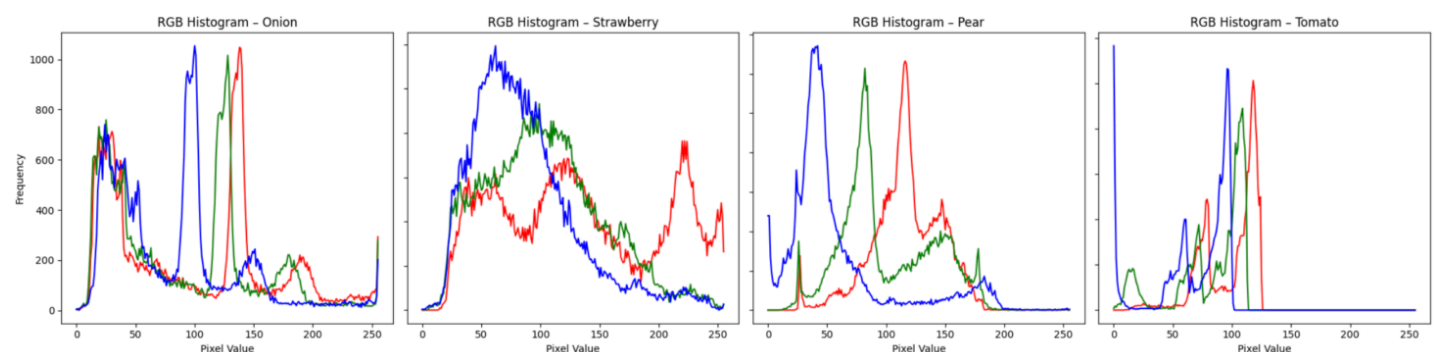
Below are sample resized images from each class (**Onion** , **Strawberry** , **Pear** , **Tomato**), illustrating the preprocessing step before feeding them into the model. For training purposes, the images were normalized by dividing each pixel value by 255.



Sample 224×224 images from each class

3.2 RGB Histogram Analysis: What It Tells Us About the Dataset

This RGB histogram plot shows the **distribution of pixel intensities** for the **Red, Green, and Blue channels** per class. It's a **visual summary of color composition** and can reveal important patterns about your dataset.



RGB histogram distribution per class

From the above histograms, we observe the following:

- **Color Signatures:** Each class has distinct RGB patterns. For example, **Tomato** shows strong red peaks; **Pear** has dominant green and blue.
- **Image Quality:** Irregular or flat histograms may indicate over/underexposed or noisy images.

- **Channel Balance:** Most classes show good **RGB variation**, so retaining **all 3 channels** is important. Onions show similar trends across **R, G, B** channels but still contain subtle distinguishing features.

Based on per-class RGB histograms, we observe the following:

1. Onion

- Red & Green: Sharp peaks at 140–150
- Blue: Broad peak around 100
- Likely reflects white/yellow onion layers with soft shadows.
- The model may learn to detect mid-range blue with red-green spikes

2. Strawberry

- Red: Two strong peaks around 80 and 220
- Green & Blue: Broader, less prominent
- Indicates dominant red intensity typical of strawberries; low blue supports lack of cool tones
- The model can distinguish this class easily due to its strong color separation

3. Pear

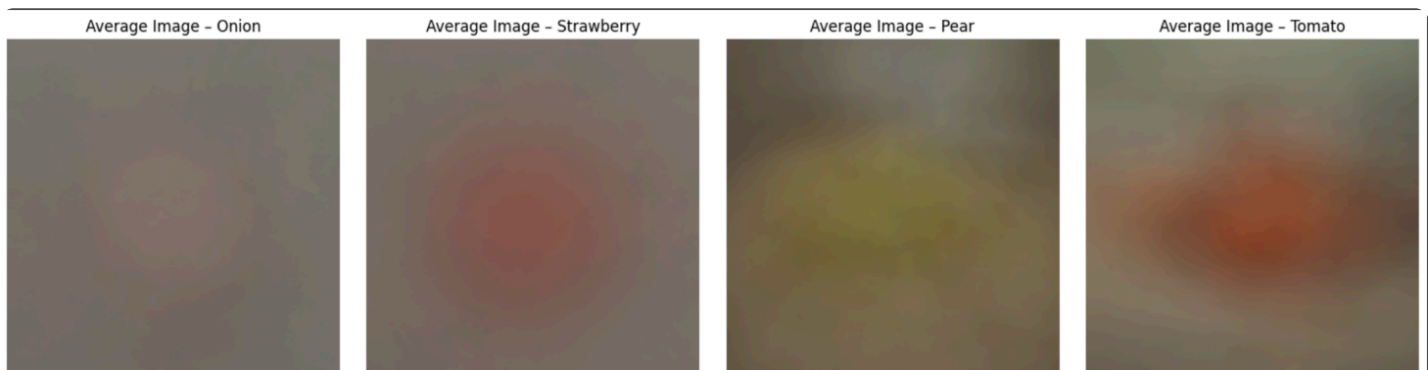
- Green & Blue: Peaks between 50–120
- Red: Moderate and spread around 100–150
- Suggests soft green/yellow pear tones with consistent lighting

4. Tomato

- Red: Very sharp peak around 120
- Green & Blue: Low and drop off quickly
- Strongly saturated red, characteristic of ripe tomatoes
- Easy for the model to detect, but caution is needed to avoid overfitting to red alone

3.3 Dataset Analysis Based on Average Images

The average images of **Onion**, **Strawberry**, **Pear**, and **Tomato** offer valuable insights into the characteristics of the dataset they were generated from. These images are created by averaging pixel values across all images in each class.



Average image per class

General Observations

1. Blurriness of All Average Images

- High blur indicates significant variation in object position, orientation, and size.
- No consistent alignment or cropping, objects appear in different parts of the frame.

2. Centered Color Blobs

- Each average image displays a dominant center color:
 - Onion: pale pinkish-grey
 - Strawberry: red core
 - Pear: yellow-green diffuse
 - Tomato: reddish-orange with brown-green
- This suggests most objects are roughly centered.
Pear and tomato are more **localized and distinct**, while onion and strawberry show more **variation and blur**.

3. Background Color and Texture

- All classes share a gray-brown tone due to a mix of background elements. As multiple colors blend, they tend to shift toward a darker gray.
- This suggests the use of natural or neutral settings with a variety of background textures and colors.

Implications for Model Training

- Color is a Strong Signal:** Average images retain dominant color, confirming the importance of **RGB input**.
- Centering Helps:** Consistent object centering allows CNNs to leverage spatial regularities.

3.4 Training and Results

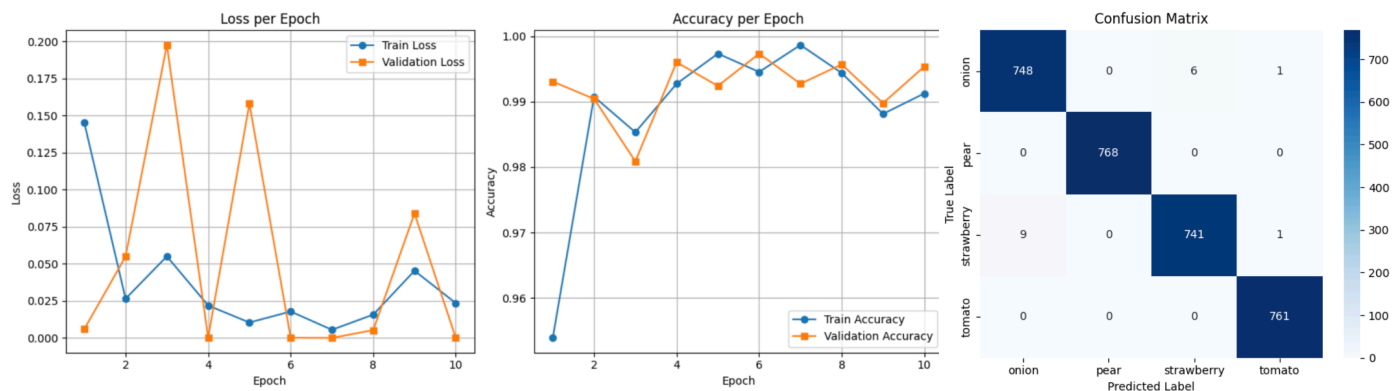
We used a dataset of **12,000 manually labeled images** covering four classes: tomato, onion, pear, and strawberry. The dataset was split in a **50:25:25 ratio** for training, validation, and testing, respectively:

- Training:** 6,000 images
- Validation:** 3,000 images
- Testing:** 3,000 images

Although the typical split is 70:15:15, we opted to test on more data to better evaluate generalization and avoid overfitting.

Due to hardware constraints (GPU memory limits), we used a **batch size of 32**. We employed the **Adam optimizer** with a learning rate of **0.0001**.

We also implemented **early stopping** with a patience of 5 epochs, meaning training stops if no improvement is seen in validation accuracy for 5 consecutive epochs.



Training vs Validation Loss and Accuracy

The model achieved over **95% accuracy within just the first epoch**.

This rapid convergence is primarily due to the use of **EfficientNet-B0**, which is pretrained on ImageNet and already contains low-level visual features.

Thanks to **transfer learning**, the model was able to learn quickly on our dataset with minimal training from scratch.

The model reached peak performance at **Epoch 6**:

- **Train Loss:** 0.0178
- **Train Accuracy:** 99.46%
- **Validation Accuracy:** 99.74%

The final **test accuracy** was **99.44%**, indicating excellent generalization to unseen data.

From the confusion matrix, it is evident that the model demonstrates strong **class separability** and **robust generalization**, with only **17 total misclassifications out of 3,035 test samples**.

This confirms that the model is capable of distinguishing these classes with high precision.

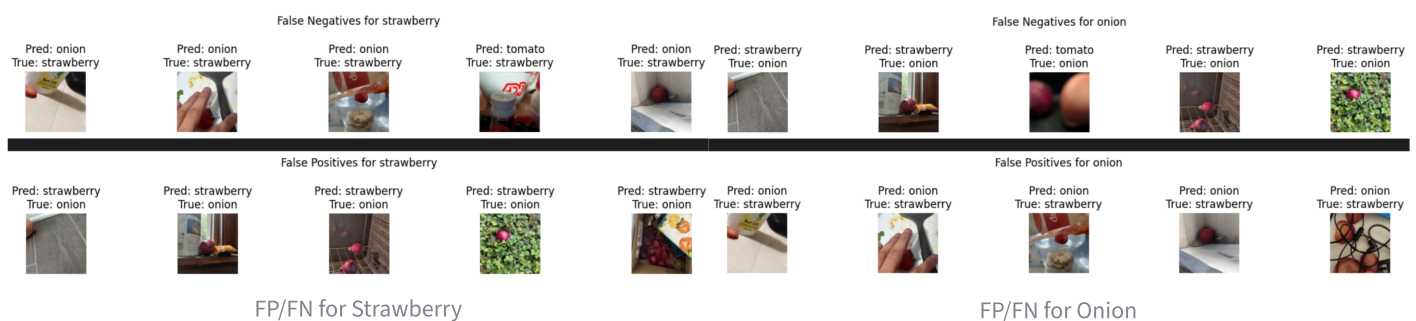
False Positives / False Negatives (Examples)

By analyzing the images that were **falsely classified** (false positives and false negatives), we can pinpoint exactly where the model is making mistakes.

These examples help us identify whether misclassifications are due to:

- Visually ambiguous or difficult samples
- Blurry or out-of-focus images
- Outliers that differ significantly from the training distribution

Reviewing these cases allows us to better understand the model's true performance and its limitations in real-world scenarios.



From the misclassified images, we can deduce that the model struggled **slightly** with images that were **out of focus**, captured in **very dim lighting**, or showed only a **small visible portion** of the object. These conditions made it difficult for the model to accurately identify the class.

Most misclassifications occurred between **strawberry** and **onion**. These classes exhibited greater variation in object positioning. In some cases, the objects (onion or strawberry) were **partially hidden**, with only a small portion visible, and were also affected by **poor lighting conditions**. Such combinations made it challenging for the model to make accurate predictions.

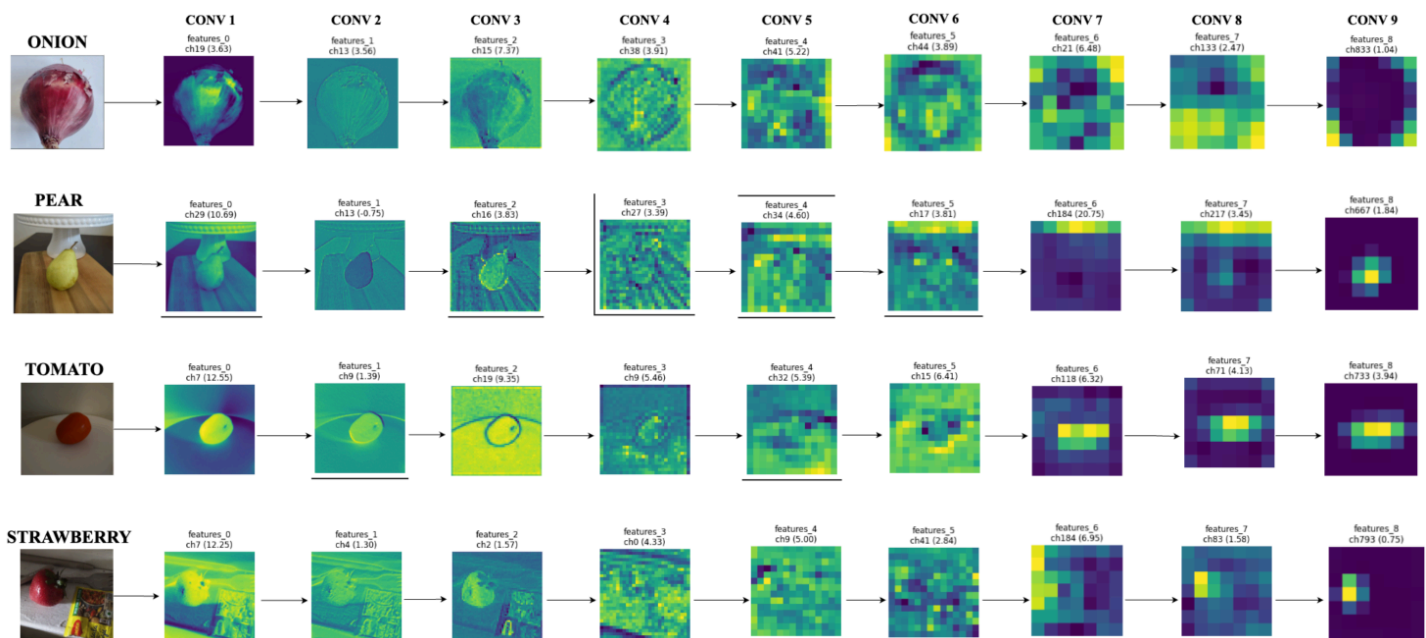
However, with an F1-score of 99% for these classes, we can confidently conclude that the model performed well overall, especially on images where the object was **clearly visible, fully within the frame**, and in **good general condition**. This further suggests that the model is **robust and ready for real-world use**.

Notably, we did not observe any misclassifications for **pear** and **tomato**. Based on our earlier data analysis, images in these classes were generally **well-centered and localized**, which likely contributed to the model's high accuracy (100%) in those categories.

Learned Feature Maps (Pattern) Analysis

To understand what our model has actually learned and how it perceives different food items internally, we visualized **feature maps** extracted from various convolutional layers of **EfficientNet-B0**.

The image below shows a **single most-activated channel per layer** for each class (Onion, Pear, Tomato, and Strawberry), across **9 convolutional stages**.



Feature maps across Conv layers for each class (EfficientNet-B0)

Each row corresponds to a different class and shows the evolution of feature extraction from **Conv1 to Conv9**, i.e., from shallow to deep layers.

1. Early Layers (Conv 1–3):

- Focus primarily on **edges, textures, and object contours**.
- All classes exhibit relatively **fine-grained spatial detail** at this stage.
- You can still visually recognize the object (e.g., the onion's round boundary or the pear's contour).
- These layers act like **edge detectors** or **low-level texture filters**.

2. Middle Layers (Conv 4–6):

- Begin to extract more **abstract, localized patterns**.

- Object boundaries start to blur, and **high-frequency detail reduces**.
- Certain class-specific structures emerge (e.g., the tomato's highlight region or the strawberry's bright patch).
- The model starts focusing on **regions of high semantic importance**.

3. Deep Layers (Conv 7–9):

- Feature maps become **coarser and more focused**, losing most spatial resolution.
- The network now highlights only **key discriminative regions**, often the **center mass** of the object.
- While the original shape is nearly lost, **strong activation in a focused area** indicates high confidence in classification.
- This shows the model is no longer looking at superficial textures, but **has learned what features truly define each class**.

Key Takeaways:

- Model **successfully learns hierarchical features**: from edges and textures to class-specific abstractions.
- The model appears to **localize the object region** consistently across all classes. Especially clear in later layers.
- This visualization confirms that the model isn't just memorizing images but is actually learning **robust visual representations** across depth.

4. Produce Variation Classification Task

As mentioned earlier, we have **3,000 images per class**, and within each class, there are **1,000 images per variation** of whole, halved/hulled, and sliced/cored.

These variations not only help make our main classification model more **robust to presentation differences**, but also allow us to analyze how the model performs under **intra-class variation**, that is, variation within the same object category.

Importance of Intra-Class Variation analysis:

- In real-world settings (e.g., cooking, grocery shelves, or user-uploaded photos), food items can appear in multiple forms (whole, cut, or partially visible).
- A model that performs well only on whole items may fail when the object is sliced or obscured.
- By training and evaluating a separate **variation classifier**, we can:
 - Assess the **distinctiveness** of each variation within a class.
 - Understand whether certain variations (e.g., "sliced onion") are harder to distinguish than others.
 - Identify **confusing cases**, which may need augmentation, re-labeling, or more data.
 - Ensure that the main classifier isn't biased toward one specific presentation.

In the following section, we train a dedicated CNN to classify the **variation type** within each produce category, and evaluate its performance across the three variation classes.

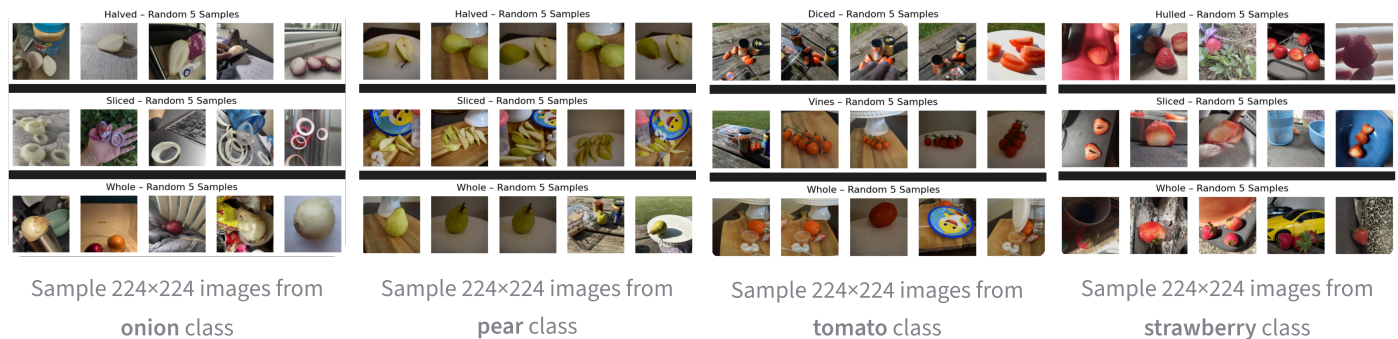
4.1 Image Preprocessing and Samples

As we are using the **EfficientNet-B0** model, all images in our dataset are resized to **224×224** pixels. This is the standard input size for EfficientNet and ensures compatibility with pre-trained weights, as well as efficient GPU utilization during training.

Below are sample resized images for each class (**onion**, **pear**, **strawberry**, and **tomato**) showing their intra-class variations: **whole**, **halved/hulled**, and **sliced**.

These samples provide a visual sense of the input data and the diversity of presentation styles within each category.

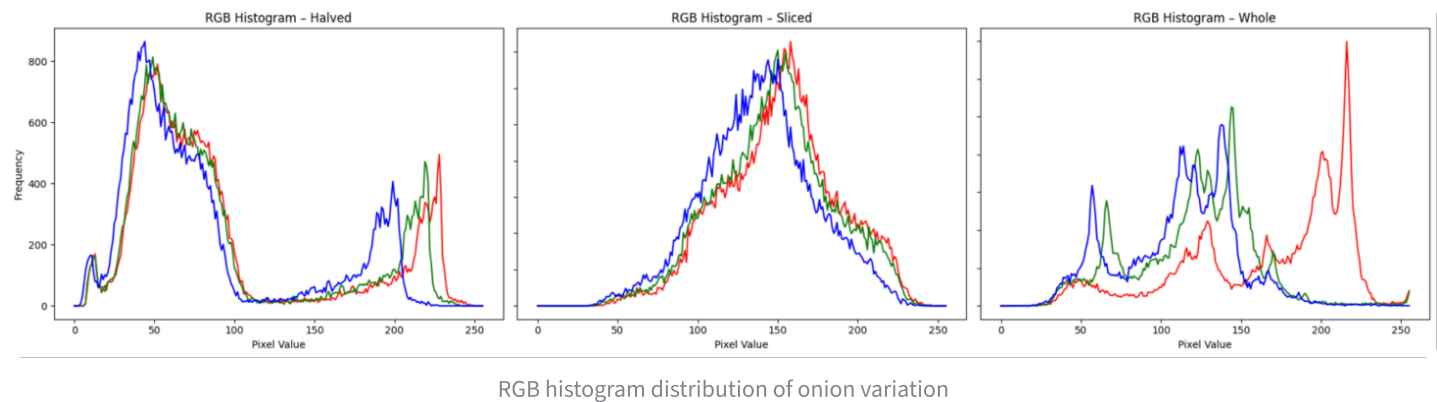
For training purposes, the images were normalized by dividing each pixel value by 255.



4.2 RGB Histogram Analysis: What It Tells Us About the Dataset

This RGB histogram plot shows the **distribution of pixel intensities** for the **Red, Green, and Blue channels** of images per class. It's a **visual summary of color composition** and can reveal important patterns about your dataset.

RGB Histogram Analysis: Onion (Intra-Class Variations)



The plots below represent RGB intensity distributions for each of the three onion variations: **Halved**, **Sliced**, and **Whole**. Each line shows pixel frequency across Red, Green, and Blue channels.

1. Halved

- **Blue channel** dominates early pixel ranges (peaks ~40–60), suggesting a bluish tint in onion layers or reflections.
- Red and Green are moderately aligned, indicating consistent lighting.
- Minor peaks at higher pixel values may result from reflective areas or background variance.
- **Interpretation:** Halved onions show strong consistency with a subtle blue tone, likely taken in well-lit but slightly cool environments.

2. Sliced

- All three channels peak around pixel values 130–150.
- Histogram is **smoother and more centered**, indicating balanced exposure and color.
- Slight red dominance in the mid-range may be due to the red/pink inner rings being more exposed.
- **Interpretation:** Sliced onions offer the most uniform and balanced appearance across all channels.

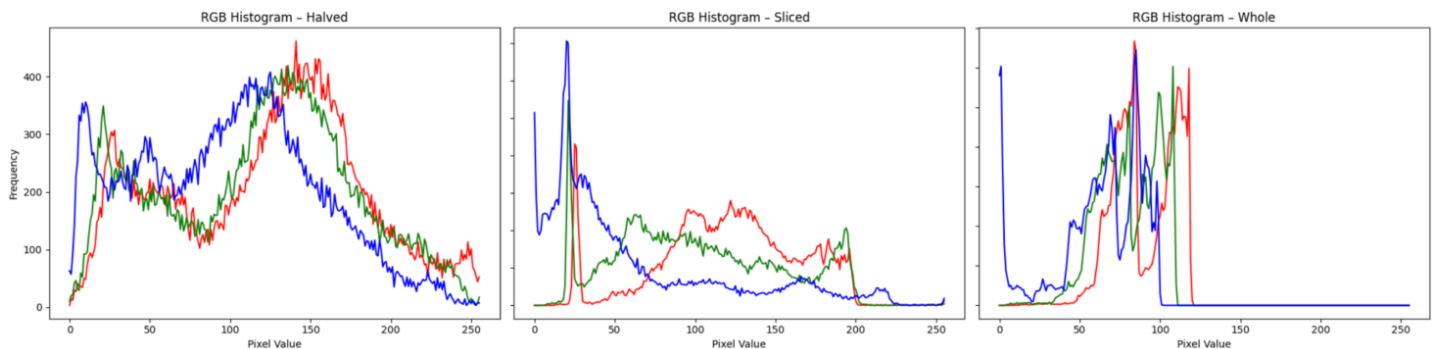
3. Whole

- Shows **high red peaks** near pixel value 220 and strong green variation around 120–150.
- Blue is less dominant and shows more fluctuation in the mid-range.
- Histogram is noisier with more channel separation, likely due to outer skin, glare, or inconsistent lighting.
- **Interpretation:** Whole onions are visually more complex, capturing skins, glare, and full curvature. This leads to higher variation.
 - To capture this complexity effectively, using **RGB channels** is essential.

Dataset Insights

- **Lighting & Background Consistency:**
 - Sliced and halved images appear more controlled and evenly lit.
 - Whole images show more **color imbalance and variation**, indicating diverse capture settings.
- **Model Implications:**
 - The model may learn **more stable features** from sliced and halved images.
 - Whole onions may require the model to rely more on **texture and shape** than color.

RGB Histogram Analysis: Pear (Intra-Class Variations)



RGB histogram distribution of Pear variation

The plots below represent RGB intensity distributions for each of the three pear variations: **Halved**, **Sliced**, and **Whole**. Each line shows pixel frequency across Red, Green, and Blue channels.

1. Halved

- Displays a **balanced and smooth distribution** across all three channels.
- Red, green, and blue channels peak around 130–150, indicating moderate brightness and natural coloration.
- No single channel dominates, which suggests good **white balance** and consistent lighting.

- **Interpretation:** Halved pears are well-exposed, and color is evenly distributed, making this variation visually clean and useful for training.

2. Sliced

- Shows a **strong blue peak at pixel value 0**, indicating the presence of **underexposed or shadowed regions**.
- Green and red are more balanced but spread across mid to high intensity values (~50–180).
- The histogram shape is more **jagged and variable**, which may suggest inconsistent lighting.
- **Interpretation:** Sliced pears may suffer from **lighting inconsistencies**, contributing to visual noise.

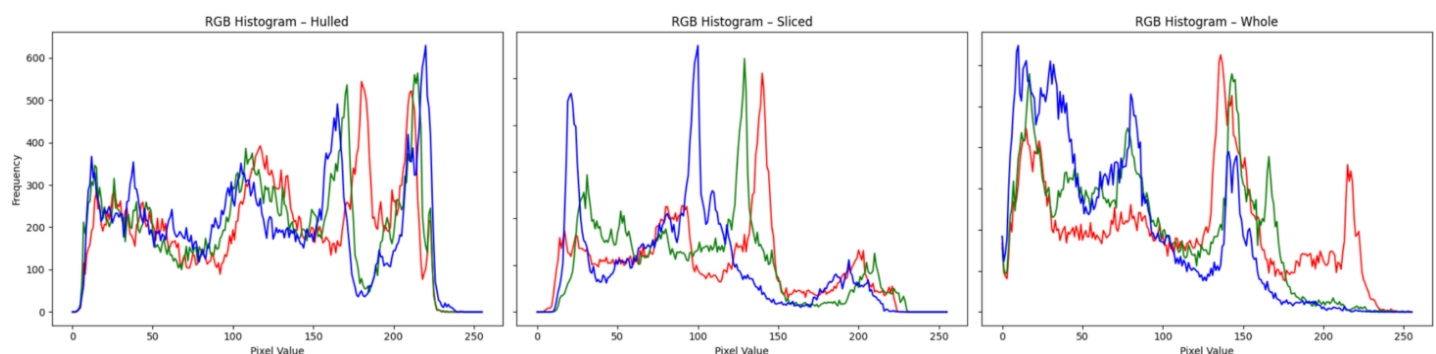
3. Whole

- RGB curves are tightly packed and peak sharply around **pixel values 80–100**, with a quick drop-off after.
- Very little spread across intensity range. Images likely have **uniform lighting** with soft shadows.
- Red channel slightly dominates.
- **Interpretation:** Whole pears appear **low in contrast and brightness**, which may simplify the learning task.

Dataset Insights

- **Lighting Conditions:**
 - Halved images show the best exposure balance.
 - Sliced images include darker regions, hinting at variability in data quality.
 - Whole pears are consistently lit.
- **Model Implications:**
 - Halved pears are optimal for training due to stable exposure.
 - Whole pears may be easily classified due to consistent appearance but may lack variation needed for generalization.

RGB Histogram Analysis: Strawberry (Intra-Class Variations)



RGB histogram distribution of Strawberry variation

The plots below illustrate the pixel intensity distributions for Red, Green, and Blue channels across three strawberry variations: **Hulled**, **Sliced**, and **Whole**. These histograms help us understand how light, color, and structure vary within the same class.

1. Hulled

- All three channels show strong, distinct peaks around **200–240**, especially red and blue, indicating the presence of **high saturation and bright highlights**.
- Moderate peaks across mid-range values (50–150) suggest good contrast.
- The histogram is **visually rich**, covering a wide range of pixel intensities.
- **Interpretation:** Hulled strawberries are well-lit and contain diverse color information, especially in the red spectrum. Good candidate for training due to high color contrast.

2. Sliced

- Displays a **strong green peak near 140** and red around 130–150, which are consistent with the **flesh and seedy outer layer** of strawberries.
- Blue is subdued across the entire range, which is expected for strawberries.
- **Interpretation:** Sliced strawberries appear more uniform and less reflective, providing a **clean color profile**.

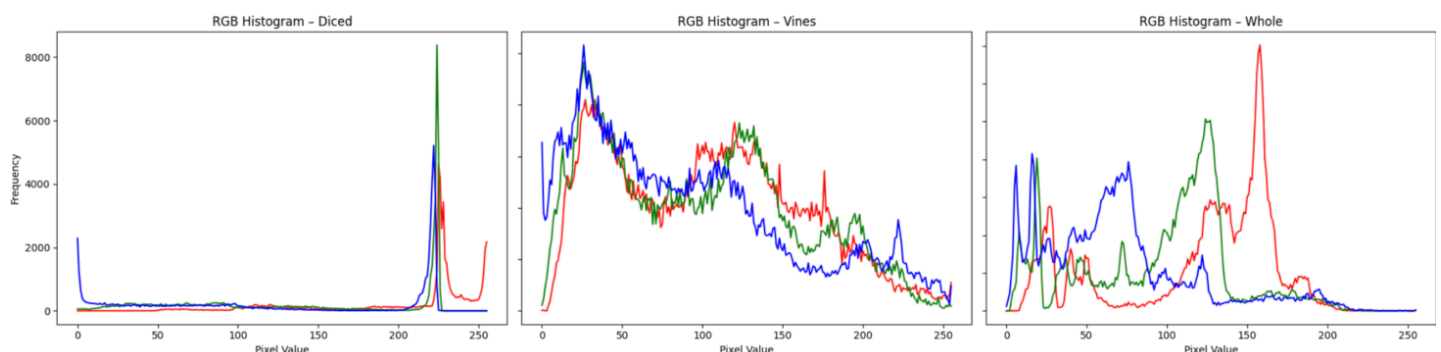
3. Whole

- Broad red and green peaks from **100–160**, with visible spikes around **140–150**, typical of a fully intact strawberry's surface.
- Blue is again low, suggesting **minimal background influence** or blue-toned lighting.
- Histogram is less spiky and more spread out than others, implying a **more natural condition**.
- **Interpretation:** Whole strawberries have balanced exposure and represent general real-world conditions well. Their diverse yet smooth histogram supports good generalization.

Dataset Insights

- **Lighting & Surface Reflection:**
 - Hulled strawberries reflect the most light, they show strong bright peaks.
 - Sliced variations are more **internal-texture dominant**, with reduced highlight intensity.
 - Whole samples offer the most **balanced histogram**, likely reflecting more consistent and natural lighting.
- **Model Implications:**
 - Each variation presents unique spectral patterns, confirming that the model can learn these differences and perform accurate classification.
 - Their differing RGB distributions also reduce the chance of model overfitting to any single presentation style.

RGB Histogram Analysis: Tomato (Intra-Class Variations)



The plots below represent RGB intensity distributions for the three tomato variations: **Diced**, **Vines**, and **Whole**. These histograms reveal how color composition and exposure vary across presentation styles.

1. Diced

- The histogram shows **sharp, narrow peaks** for all three channels near pixel values **230–250**, indicating **high saturation and brightness** — possibly due to light reflection from diced surfaces.
- A significant spike in the **blue channel at pixel 0** suggests underexposed or shadowed areas, likely from the background.
- Minimal spread across the mid-tone range (50–200) implies **low color diversity**.
- **Interpretation:** Diced tomatoes contain bright highlight, with limited mid and low-tone information. This variation could confuse the model in **inconsistent lighting**, but it performs well under controlled lighting. It also suggests lower variation across images.

2. Vines

- Displays a **broad, balanced distribution** across all channels, especially strong in the blue and green spectrum (~20–150).
- No strong spikes, suggesting **natural, diffuse lighting** and less glossiness.
- Color spread across all pixel values shows **greater background diversity**, possibly due to the inclusion of leaves, stems, or soil.
- **Interpretation:** Vines are visually complex and rich in texture, offering the **highest visual diversity** among the three. These images reflect realistic environments.

3. Whole

- Strong **red peak near 150–160** represents the core tomato surface.
- Green and blue show defined peaks around 90–130, suggesting presence of both background and stem/leaf regions.
- Well-defined, multi-peak structure shows moderate saturation and **good contrast**.
- **Interpretation:** Whole tomatoes appear cleanly illuminated and well-captured, with a **balanced mix of object and background**.

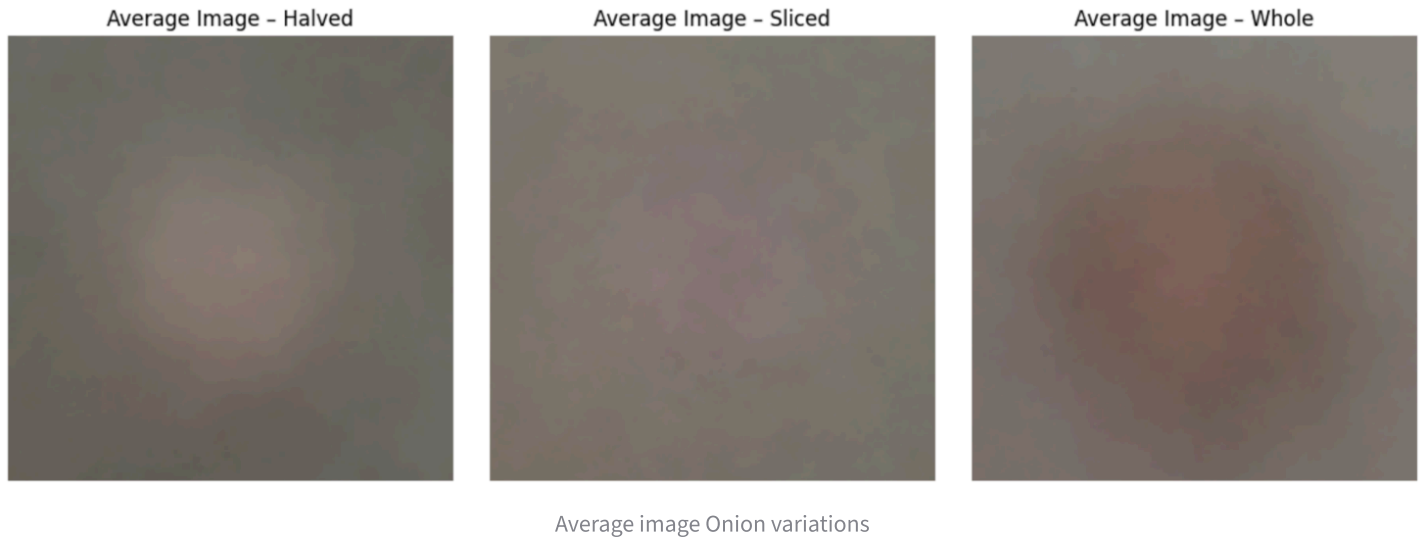
Dataset Insights

- **Lighting & Background:**
 - Diced tomatoes show high extremes of highlights, likely affected by direct light.
 - Vines exhibit diffuse lighting but introduce **non-tomato color features**.
 - Whole images appear most balanced and consistent in lighting and color spread but lack high brightness data.
- **Model Implications:**
 - Each variation brings complementary features: diced emphasizes color intensity, vines offer real-world complexity, and whole provides consistency.
 - Lack of variation may hinder generalization under challenging conditions, but it can perform well in good lighting.

4.3 Dataset Analysis Based on Average Images

The average images of intra class variations offer valuable insights into the characteristics of the dataset they were generated from. These images are created by averaging pixel values across all images in each class.

Average Image Analysis: Onion (Intra-Class Variations)



Visual Observations

1. Halved

- Very high blur, suggesting **large variation** in object orientation and placement.
- Likely a mix of different halves (top/bottom) with varied alignment.

2. Sliced

- Slightly more blurry than halved, might be caused by reason as sliced onion rings take up very less space in image.
- Faint radial patterns hint at partial consistency in shape, which model can learn.

3. Whole

- Most distinct shape and color among the three.
- Central reddish blob is clearly visible.
- Indicates strong consistency in pose, orientation, and background.

Implications for Modeling

- **Whole:**

- High consistency makes it easier for models to learn.
- Ideal for CNNs, as it's well-centered and uniformly structured objects.

- **Sliced & Halved:**

- Require additional preprocessing or augmentation.
- **Higher intra-class variation** may lower model performance if not addressed.

Average Image Analysis: Pear (Intra-Class Variations)



Visual Observations

1. Halved

- The soft yellow-green blob is relatively centered but very diffused.
- This suggests that while objects are roughly centered, their orientation, scale, and cropping vary significantly.

2. Sliced

- The yellow region is more centralized and denser than in the halved class, indicating better consistency in object placement across samples.
- However, the blur indicates that slices still vary in size, number, and arrangement.

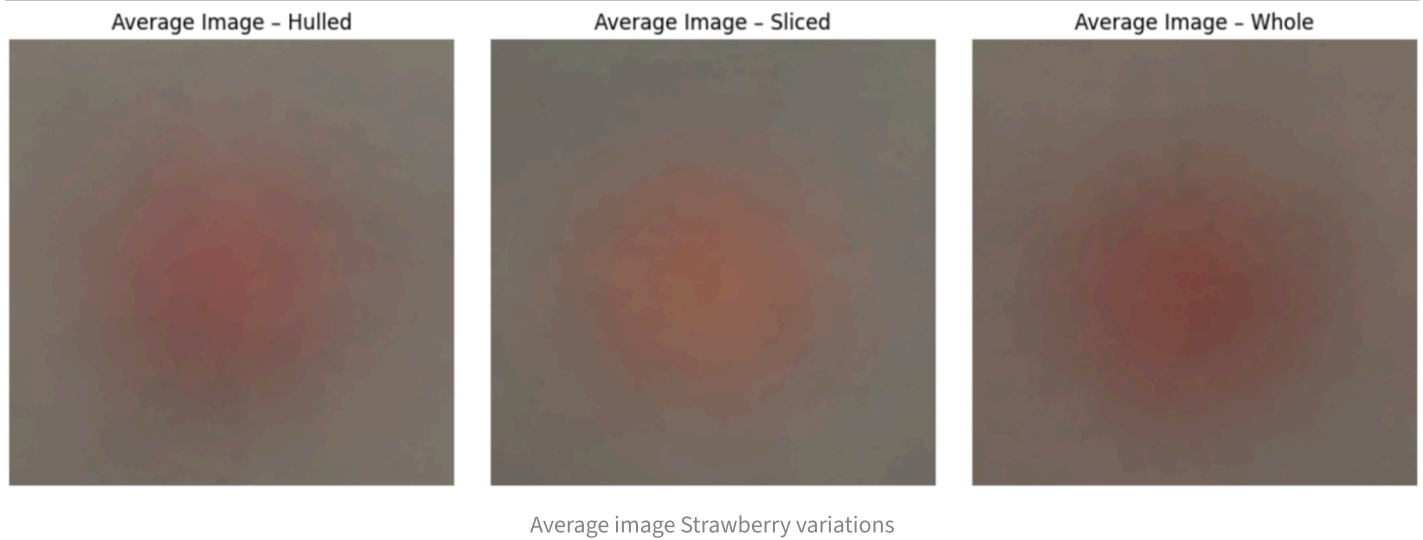
3. Whole

- The bright yellow-green blob is the most prominent and sharply centered.
- Strong evidence of consistent centering, scale, and posture.
- Least blur, indicating high uniformity across samples. which may lead to overfitting and reduced generalization.

Implications for Modeling

- All three classes appear to be roughly centered, which means the model might struggle with challenging or unusual positioning.

Average Image Analysis: Strawberry (Intra-Class Variations)



Visual Observations

1. Hulled

- The average image has a compact red blob at the center.
- This indicates that most hulled strawberries are consistently centered and aligned.

2. Sliced

- The average image appears more orange and diffuse compared to hulled.
- This suggests a higher variation in slice count, thickness, or arrangement.
- The blur shows the slices are still mostly centered but vary in shape and coverage.

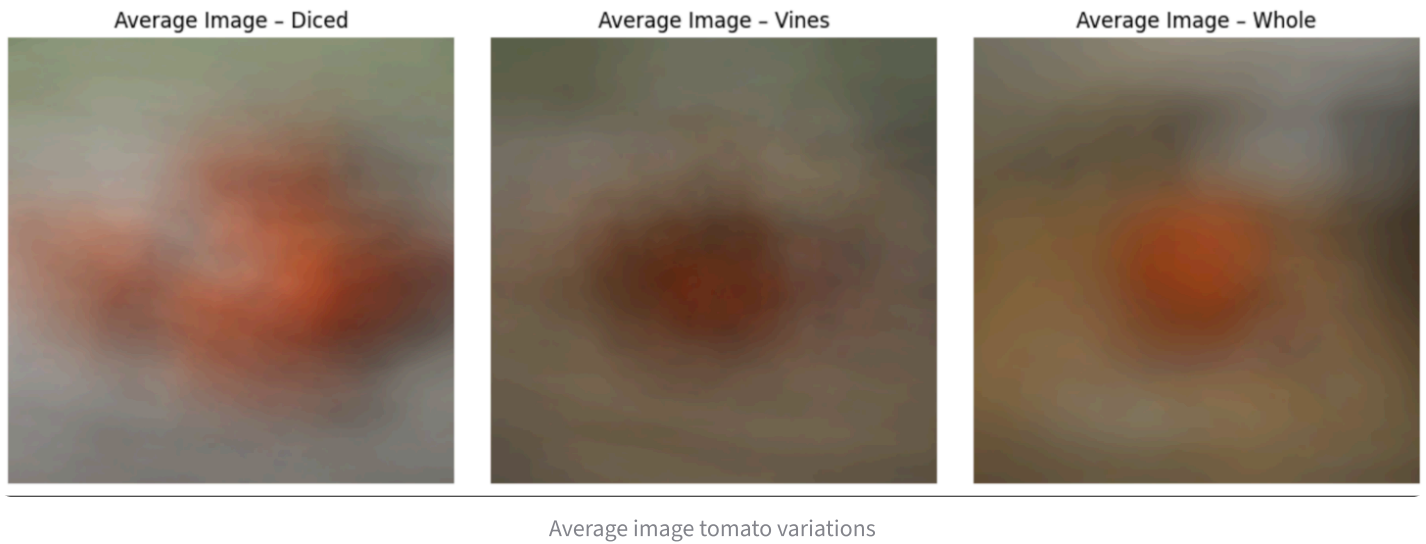
3. Whole

- The average image shows a slightly darker, rounder red blob than hulled.
- It is well-centered and more uniform than sliced.
- Indicates some variation in pose or camera angle, but overall still consistent around center.

Implications for Modeling

- All three categories have well-centered objects and also show some variation in position. This makes it easier for models to learn and extract features, especially due to the consistent central positioning, while also enabling learning under challenging conditions.

Average Image Analysis: Tomato (Intra-Class Variations)



Visual Observations

1. Diced

- Multiple reddish blobs are visible but still form a centralized mass.
- This indicates that diced tomatoes, while individually small, are often grouped toward the center across samples.
- Shows moderate variation in shape and number, but not in placement.

2. Vines

- A distinct red cluster appears at the center, surrounded by subtle textures.
- This suggests tomatoes on vines are generally centered, but with extra visual components (stems, leaves) adding background complexity.
- Moderate blur indicates some variability in orientation and scene layout.

3. Whole

- A very defined and uniform circular red blob is present at the center.
- Suggests consistent centre alignment and pose across images.

Implications for Modeling

- All three classes exhibit **strong central alignment**. For better generalization, it is preferable that the central blob appears more blurred, as this indicates diverse object positioning and conditions, which helps the model perform well across varied scenarios.
- Despite differences in object structure, the consistent central positioning across all three classes allows models to effectively learn spatially anchored features, but may cause the model to struggle when objects appear in different positions or when the image and its background is complex.

4.4 Image Analysis conclusion

The combination of average images and RGB histogram plots reveals that, in general, all classes (onion, pear, strawberry, tomato) demonstrate a strong central focus in their average images. This is ideal for convolutional neural networks

(CNNs), which exploit spatial locality. However, such consistency may limit generalization to real-world, off-centered samples.

Insights by Visual Feature

Feature	Observation	Implication
Positioning	Most images are well-centered across all classes and variations.	Models will learn easily but may fail under real-world misalignment.
Blur in Average Images	Indicates intra-class variation. More blur = more diversity in orientation, size, and structure. Onion and strawberry show the most blur among intra-class variations.	Helps generalization if balanced; too little variation risks overfitting.
Histogram Shape	Distinct peaks indicate color dominance and exposure levels. It's observed consistently across all classes.	Highlights the importance of using RGB channels as input.
Background Consistency	Pear and tomato classes tend to have cleaner backgrounds.	Models trained on these may struggle with background clutter in real-world scenarios.

While the dataset provides clean and centered images conducive to initial model training, there is a **risk of overfitting to ideal conditions**.

4.5 Training and Results

We used a dataset of **3,000 manually labeled images per class**, with 1,000 images for each intra-class variation (**whole**, **halved/hulled**, and **sliced**) across four categories: **tomato**, **onion**, **pear**, and **strawberry**.

The dataset was split using either a **60:20:20 ratio** or, in some cases, a **50:25:25 ratio** for training, validation, and testing, respectively.

Onion & Strawberry

- **Training:** 1,500 images → **6,000 after augmentation**
- **Validation:** ~700 images
- **Testing:** ~700 images
- **Optimizer:** Adam
- **Learning Rate:** 0.0001

Pear & Tomato

- **Training:** 1,827 images
- **Validation:** ~600 images
- **Testing:** ~600 images
- **Optimizer:** Adam
- **Learning Rate:** 0.0001

For **strawberry** and **onion**, we applied data augmentation using image rotations (90°, 180°, and 270°), increasing the training samples from **1,500 to 6,000**, while keeping the **validation and test sets at approximately 700 images** each.

Why augmentation for onion and strawberry?

From the average image analysis, these classes showed **higher visual noise and blur**, indicating significant intra-class variation. Without augmentation, the model risked **overfitting to noise** and generalizing poorly. Rotation-based augmentation helped expose the model to diverse orientations and reduce this risk.

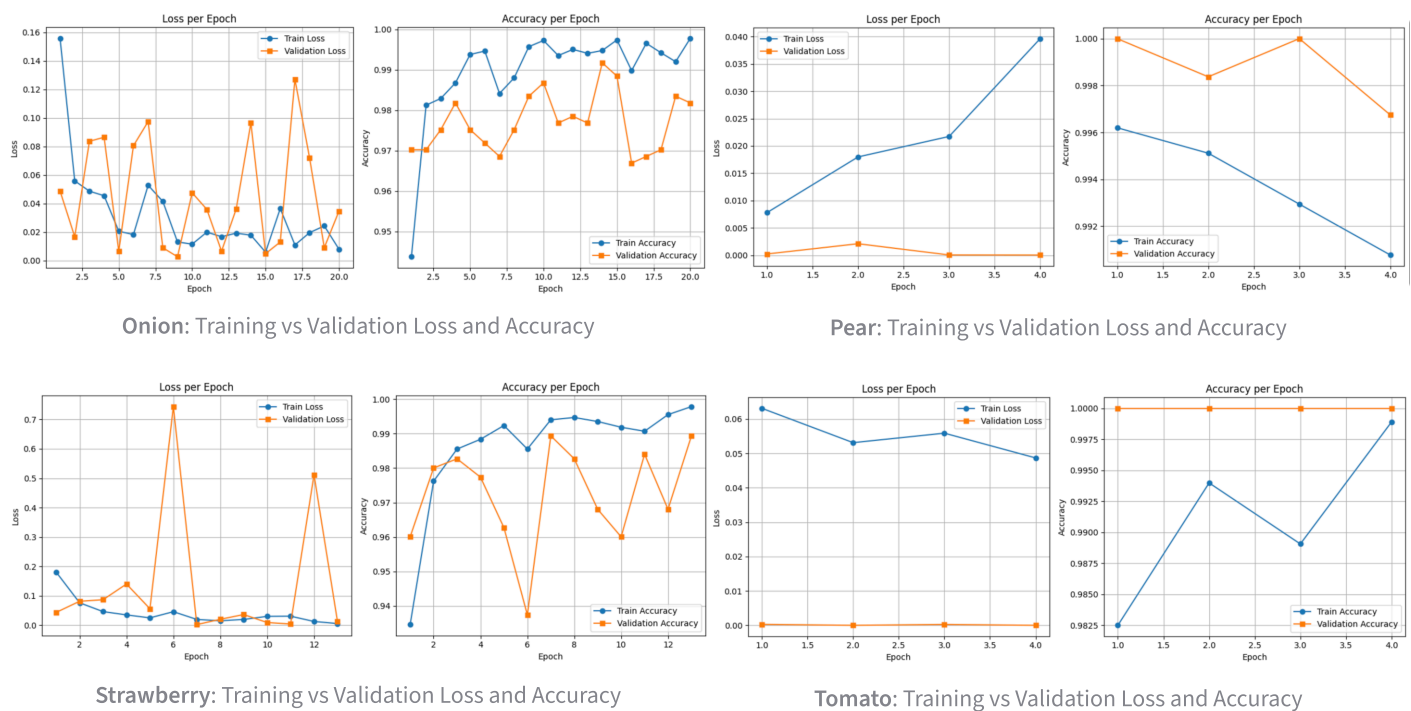
Why no augmentation for pear and tomato?

Our analysis of their average images and RGB histograms revealed that these classes were **well-centered**, **well-lit**, and had **limited background variation**. As a result, the model could learn from them effectively without augmentation. Although classification performance may degrade in real-world scenarios with cluttered or complex backgrounds, in **ideal settings** where images are centered and consistently lit, these classes are expected to yield **strong performance even without augmentation**.

Due to hardware constraints (GPU memory limits), we used a **batch size of 32**. We also implemented **early stopping** with a patience of 3 epochs, meaning training stops if no improvement is seen in validation accuracy for 3 consecutive epochs.

Model Performance Summary

We saved the model with the **highest validation accuracy** and the **smallest difference between training and validation accuracy** to avoid any form of overfitting.



The above graph shows the training and validation accuracy and loss curves. We can observe that **Pear** and **Tomato** reached 100% validation accuracy within **one** epochs, whereas **Onion** and **Strawberry** took longer to achieve high accuracy.

Despite the initial differences, all models eventually reached **very good performance**, and their detailed classification reports are provided below.

Onion • Epoch: 14 • Train Loss: 0.0179 • Train Accuracy: 99.48% • Validation Accuracy: 99.17% • Test Accuracy: 98.68% Classification Report • Halved: Precision 0.99, Recall 0.99, F1-score 0.99 • Sliced: Precision 0.99, Recall 0.98, F1-score 0.99 • Whole: Precision 0.98, Recall 0.99, F1-score 0.99	Pear • Epoch: 01 • Train Loss: 0.0078 • Train Accuracy: 99.62% • Validation Accuracy: 100.00% • Test Accuracy: 100.00% Classification Report • Halved: Precision 1.00, Recall 1.00, F1-score 1.00 • Sliced: Precision 1.00, Recall 1.00, F1-score 1.00 • Whole: Precision 1.00, Recall 1.00, F1-score 1.00
Strawberry • Epoch: 07 • Train Loss: 0.0192 • Train Accuracy: 99.40% • Validation Accuracy: 98.93% • Test Accuracy: 98.67% Classification Report • Hulled: Precision 0.99, Recall 0.97, F1-score 0.98 • Sliced: Precision 0.99, Recall 0.99, F1-score 0.99 • Whole: Precision 0.98, Recall 1.00, F1-score 0.99	Tomato • Epoch: 01 • Train Loss: 0.0631 • Train Accuracy: 98.25% • Validation Accuracy: 100.00% • Test Accuracy: 100.00% Classification Report • Diced: Precision 1.00, Recall 1.00, F1-score 1.00 • Vines: Precision 1.00, Recall 1.00, F1-score 1.00 • Whole: Precision 1.00, Recall 1.00, F1-score 1.00

Model Report

All models achieved very high accuracy:

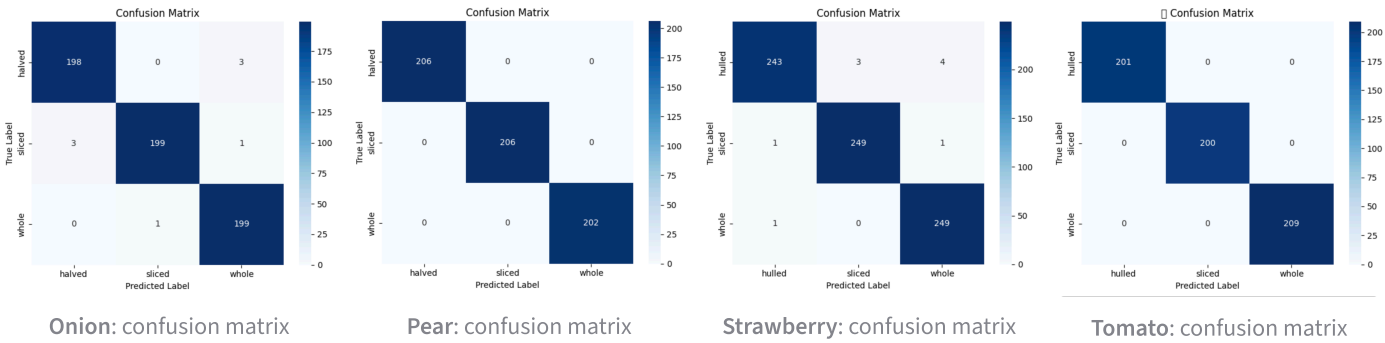
- **Pear & Tomato:** 100% test accuracy
- **Onion & Strawberry:** ~98.6% test accuracy

These results show that the dataset was clean, well-labeled, and had consistent object placement. However, 100% test accuracy may indicate lack of real-world complexity in the test set.

Overall: The models perform extremely well under ideal conditions.

False Positives / False Negatives

The confusion matrix provides insights into how our model performed, including whether it made any misclassifications and, if so, between which classes the confusion occurred.



1. Onion

The model shows strong overall performance but made a few misclassifications between **halved** ↔ **whole** and **sliced** ↔ **halved**, suggesting slight confusion due to visual similarity in edge cases.

2. Strawberry

Minor confusion is observed between **hulled** and **whole**, likely due to similar color and shape. Still, the model maintains excellent overall accuracy and balance.

3. Pear

Perfect classification across all classes (no false positives or false negatives), reflecting highly consistent, separable visual features in the dataset.

4. Tomato

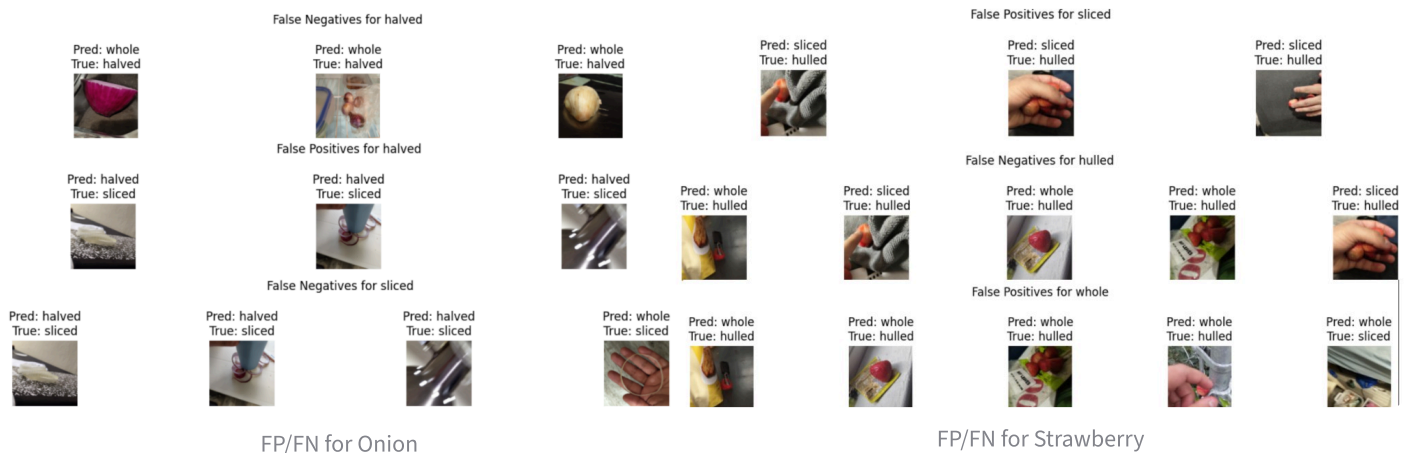
No misclassifications were made. The model distinguishes **diced**, **sliced**, and **whole** tomatoes perfectly, likely due to strong shape and texture differences across classes.

Visual Analysis of FN/FP

We know that there are no misclassifications for **pear** and **tomato**, but there are some for **strawberry** and **onion**.

By examining the misclassified images, we can determine whether these are edge cases or visually complex examples.

This helps us understand whether the model has learned the important features or is also misclassifying simple, obvious images.



The FP/FN examples for **onion** and **strawberry** reveal that the model often struggles with **borderline or visually ambiguous cases**. These cases can be a bit ambiguous for humans as well.

- For **onions**, many misclassified examples involve **poor lighting**, **background clutter**, or **partial views** of the object (e.g., close-up or occluded views of halved onions).
- For **strawberries**, the model tends to confuse **hulled** and **sliced** variants. This likely happens due to **similar color/texture**, especially when slicing is mistaken for the top of a hulled image. Some misclassified examples also show **hands or objects in the frame**, indicating that **background noise affects classification**.

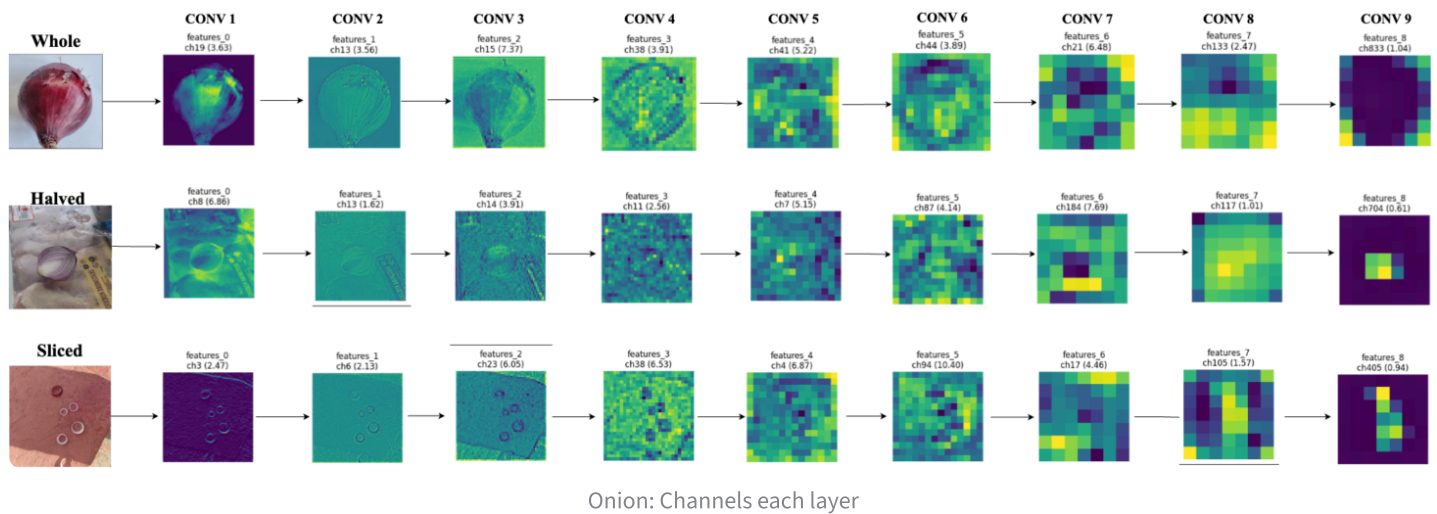
Overall, these misclassifications imply that the model performs well on clean, canonical examples but may falter under variation in lighting or occlusion.

Learned Feature Maps (Pattern) Analysis

To understand what our model has actually learned and how it perceives different food items internally, we visualized **feature maps** extracted from various convolutional layers of **EfficientNet-B0**.

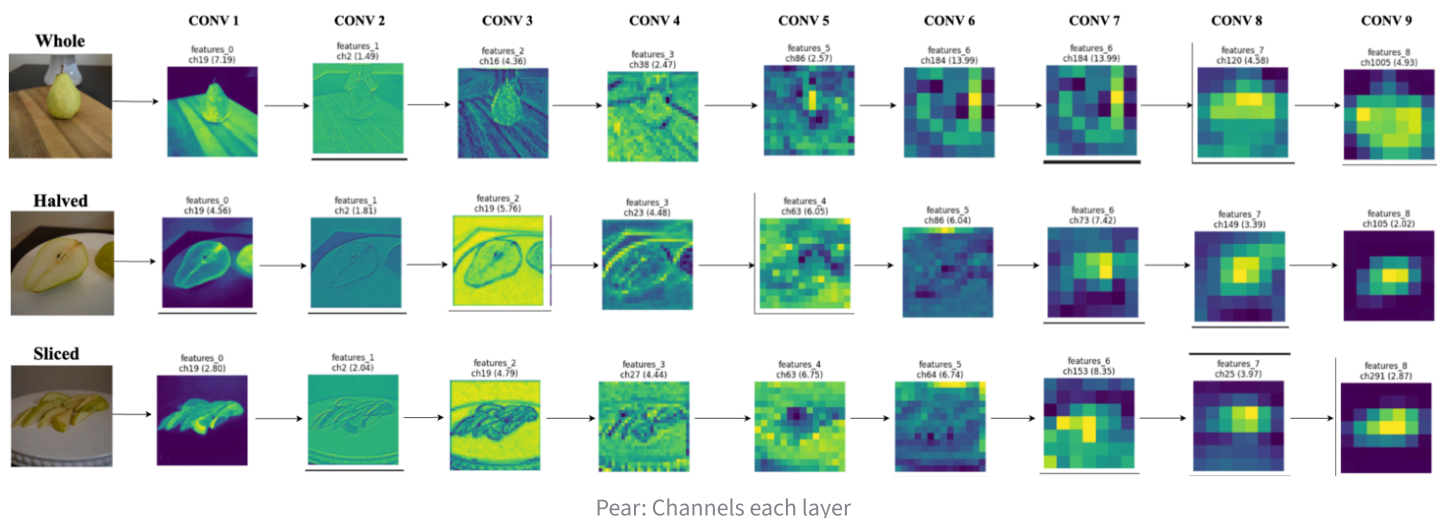
The image below shows the **single most-activated channel per layer** for each intraclass variation (whole, halved/hulled, and sliced) of the main classes: Onion, Pear, Tomato, and Strawberry, across **9 convolutional stages**.

Onion Intra-class Map Analysis



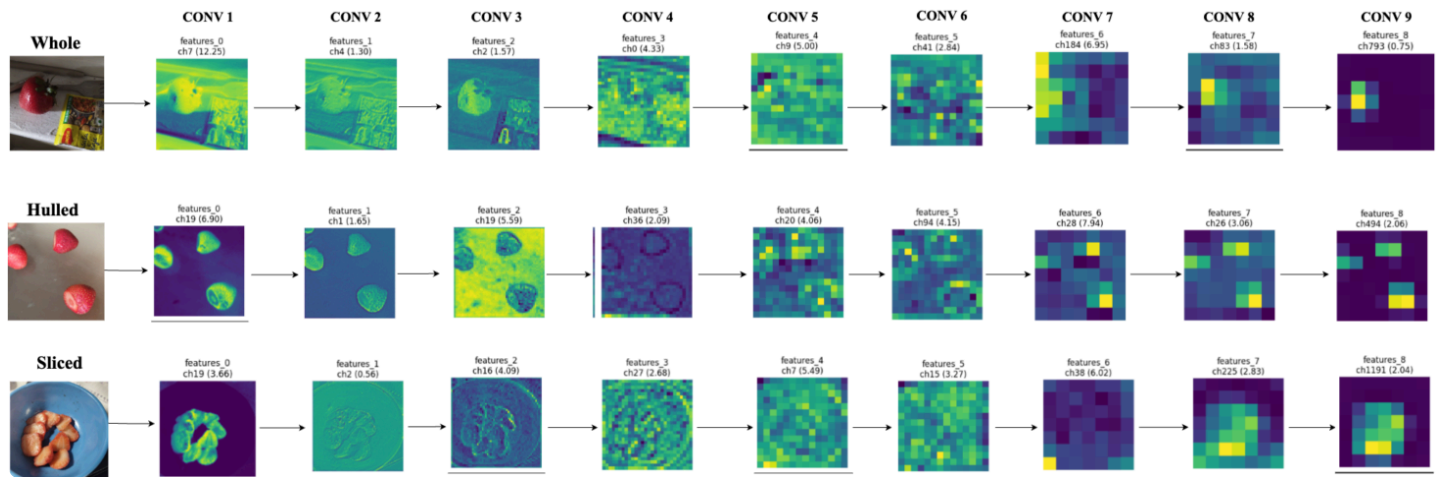
- **Whole:** Initial layers clearly capture the round bulb shape and strong edge details. As we move deeper, the model focuses on inner textures and center activation.
- **Halved:** Earlier layers detect circular contours well. Deeper layers show more dispersed activations.
- **Sliced:** Earlier layers isolate circular ring patterns effectively. Later layers show more defined and strong central activations.

Pear Intra-class Map Analysis



- **Whole:** Early layers highlight the pear shape and lighting edges. Strong attention is given to the vertical body. Later layers retain this spatial focus.
- **Halved:** Feature maps capture the internal seed cavity and split texture effectively. Consistent center-focused activation is observed.
- **Sliced:** Although flat in shape, sliced pears still maintain good feature flow. It also shows center-focused activation.

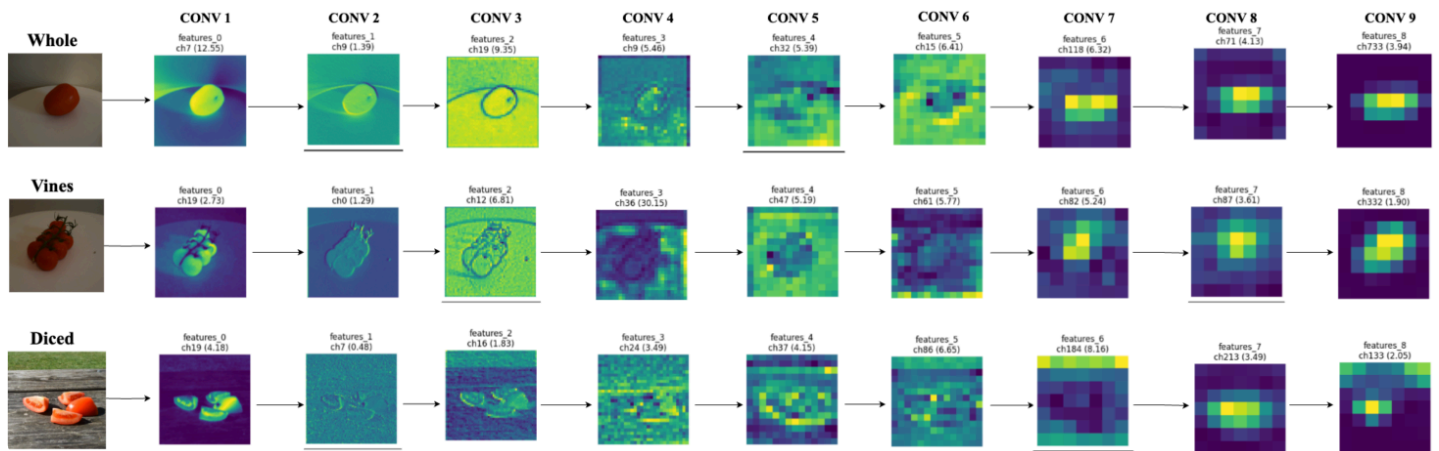
Strawberry Intra-class Map Analysis



Strawberry: Channels each layer

- **Whole:** Attention on object edges and shadows in early layers. Later activations begin to focus more on object.
- **Hulled:** Strong and crisp focus on object boundaries across all layers. Highlights strawberry contours and texture clearly.
- **Sliced:** Recognizes inner structure and scattered placement. Centralized patches persist in deeper layers, indicating successful encoding of sliced textures.

Tomato Intra-class Map Analysis



Tomato: Channels each layer

- **Whole:** High activation on elliptical shape and color gradient. Mid and deep layers preserve tomato body well.
- **Vines:** Early layers capture fine vine structures and deeper layers focuses well on the object.

- **Diced:** Early stages show multiple sharp activations on cut surfaces. Later stages focus cleanly on central parts with well-formed feature blocks.

Key Takeaways:

- The model **adaptively recognizes visual patterns** across variations within the same class.
- It **leverages shape consistency and repetitive textures** (like rings, seeds, or slices) for confident predictions.
- It handles **cluttered or occluded cases** reasonably well but shows minor degradation when the context becomes noisy (e.g., packaging, hands, lighting shadows).

This reinforces that while the model is robust to moderate noise, it **performs best on clean, centered, and clearly structured examples**. It also suggests that our dataset is mostly composed of clean, centered images with a few complex variations.

5. NLP Pipeline

5.1 Data Sources:

The project draws from two CSV files:

- **Raw_recipes.csv:** 231,637 rows, one per recipe with columns: *id, name, ingredients, tags, minutes, steps, description, n_steps, n_ingredients*
- **Raw_interactions.csv:** user feedback containing *recipe_id, user_id, rating, review text*

5.2 Corpus Filtering and Subset Selection

- **Invalid rows removed:** recipes with empty ingredient lists, missing tags, or fewer than three total tags
- **Random sampling:** 15,000 recipes selected for NLP fine-tuning
- **Positive/negative pairs:** generated for contrastive learning using ratings and tag similarity
- **Train/test split:** 80/20 stratified split (12,000/3,000 pairs)

5.3 Text Pre-processing Pipeline

- **Lower-casing & punctuation removal:** normalized to lowercase, special characters stripped
- **Stop-descriptor removal:** culinary modifiers (*fresh, chopped, minced*) and measurements (tablespoons, teaspoons, cups, etc.) removed
- **Ingredient ordering:** re-ordered into sequence: protein → vegetables/grains/ dairy → other
- **Tag normalization:** mapped to 7 main categories: *cuisine, course, main-ingredient, dietary, difficulty, occasion, cooking_method*
- **Tokenization:** standard *bert-base-uncased* WordPiece tokenizer, sequences truncated/padded to 128 tokens

5.4 Technical Specifications

Dataset:

- Total Recipes: 231,630

Infrastructure:

- Python 3.10

- Training Set: 12,000 recipes
- Average Tags per Recipe: ~6
- Ingredients per Recipe: 3-20
- PyTorch 2.1 (CUDA 11.8)
- Transformers 4.38
- Google Colab A100 GPU

5.5 Model Architecture

- **Base Model:** bert-base-uncased
- **Additional Layers:** In some runs, we added a single linear classification layer with dropout ($p = 0.1$)
- **Training Objective:** Triplet-margin loss with margin of 1.0

We trained the model directly on the raw data to see if we will get any good results. As seen in table 1, this run resulted in a very low training error but when ran on the validation set, the training error was higher. We then used cleaned up the data by removing any empty space, standardized to lower text, removed all punctuation and retrained the model. This resulted in a highly overfitted model as seen in table 1 and the results section below. Next, we added a single linear layer on top of the BERT's current architecture and added a dropout to get rid of overfitting. The results as shown in table 1 were better. Although the semantic results were better than before, it still was not good in indentifying the relationships between ingredients and the different tags. We then further structured the data by ordering the tags and ingredients in a strcutured manner across the dataset and retrained the model. This resulted in a better training and validation loss. This is also evident in the semantic retrieval results below.

5.6 Hyperparameters and Training

- **Batch size:** 8
- **Max sequence length:** 128 tokens
- **Learning rate:** 2×10^{-5}
- **Weight decay:** 0.01
- **Optimizer:** AdamW
- **Epochs:** 3
- **Hardware:** Google Colab A100 GPU (40 GB VRAM)
- **Training time:** ~30 minutes per run

Mathematical Formulations and Top-K Retrieval

Query Embedding and Similarity Calculation: we used the trained model weights to generate embeddings for the entire recipe corpus. We then used cosine similarity to calculate the similarity between the query and the recipe corpus. and once the user query is passed, we embedded the query using the trained model and used the cosine similarity formula below to retrieve the top-K recipes. We then filtered the only ones that have an average rating ≥ 3.0 and at least 5 ratings. We then sorted the recipes by similarity and then by average rating.

$$\text{Similarity}(q, r_i) = \cos(\hat{q}, \hat{r}_i) = \frac{\hat{q} \cdot \hat{r}_i}{\|\hat{q}\| \|\hat{r}_i\|}$$

Where $\hat{q}$ is the BERT embedding of the query, and $\hat{r}_i$ is the embedding of the i-th recipe.

5.7 Results

Training and Validation Loss

	Run	Configuration	Epoch-3 Train Loss	Validation Loss
0	1	Raw, no cleaning/ordering	0.0065	0.1100
1	2	Cleaned text, unordered	0.0023	0.0000
2	3	Cleaned text + single layer + dropout	0.0061	0.0118
3	4	Cleaned text + ordering	0.0119	0.0067

Table 1: Training and Validation Loss for each run

Key Finding: Run 4 (cleaned text + ordering) achieved the best balance between low validation loss and meaningful retrieval quality.

Qualitative Retrieval Examples

In this section, we will show how the results of the model differ between runs and how the model performs on different queries.

Query: "beef steak dinner"

- Run 1 (Raw): *to die for crock pot roast, crock pot chicken with black beans*
- Run 2 (Cleaned text, unordered): *aussie pepper steak steak with creamy pepper sauce*
- Run 3 (Cleaned text + single layer + dropout): *balsamic rib eye steak with bleu cheese sauce*
- Run 4 (Final): *grilled garlic steak dinner, classic beef steak au poivre*

Query: "chicken italian pasta"

- Run 1 (Raw): *to die for crock pot roast, crock pot chicken with black beans*
- Run 2 (Cleaned text, unordered): *baked chicken soup*
- Run 3 (Cleaned text + single layer + dropout): *absolute best ever lasagna*
- Run 4 (Final): *creamy tuscan chicken pasta, italian chicken penne bake*

Query: "vegetarian salad healthy"

- Run 1 (Raw): *to die for crock pot roast*
- Run 2 (Cleaned text, unordered): *avocado mandarin salad*
- Run 3 (Cleaned text + single layer + dropout): *black bean and sweet potato salad*
- Run 4 (Final): *kale quinoa power salad, superfood spinach & berry salad*

5.8 Discussion

The experimental evidence underscores the importance of disciplined pre-processing when adapting large language models to niche domains. The breakthrough came with ingredient-ordering (protein → vegetables → grains → dairy → other) which supplied consistent positional signals. As we can see in the results, the performance of the model improves with the addition of the single layer and dropout but the results are still not as good as the final run where we added the ordering of the ingredients.

Key Achievements:

- End-to-end recipe recommendation system with semantic search
- Meaningful semantic understanding of culinary content
- Reproducible blueprint for domain-specific NLP applications

Limitations:

- Private dataset relatively small training set (12k samples) compared to public corpora
- Further pre-processing could be done to improve the results
- Minimal hyperparameter search conducted
- Single-machine deployment tested
- The model is not able to handle complex queries and it is not able to handle synonyms and antonyms.

6. Website Development

- We used streamlit to develop the website. However, we faced few issues with the size of the trained model and we switched hosting to Hugging Face.
- The website loads the pre-trained models and asks the user to add any picture to classify it.
- The website also loads the recipes embeddings and top-k retrieval function and waits for the user to enter a query.
- The query is then processed by the model and top-k recipes are returned.
- All the findings and results are displayed in the report section of the website and is also available for download.

7. References

- [1] Vaswani et al., "Attention Is All You Need," NeurIPS, 2017.
- [2] Devlin et al., "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," NAACL-HLT, 2019.
- [3] Reimers and Gurevych, "Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks," EMNLP-IJCNLP, 2019.
- [4] Hugging Face, "BERT Model Documentation," 2024.
- [5] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," ICML, 2019.

© 2025 CSE 555 Term Project. All rights reserved.

Made with ❤️ by Saksham & Zaher | CSE555 @ UB